

20250730日报

今日工作内容

1. 学习vtprotobuf，并进行测试；
 - a. vtprotobuf是一个基于protoc的插件（protoc-gen-go-vtproto），与标准protoc-gen-go协同工作，生成高度优化的辅助代码，其核心目标是避免反射操作和减少内存分配，通过编译时预生成高效代码，替代运行时反射，从而提升性能；
 - b. 标准Protobuf库依赖反射动态解析消息结构，而vtprotobuf在编译时为每个消息生成定制化的序列化/反序列化代码，直接操作内存布局，避免反射开销；
 - c. vtprotobuf几个核心的优化方法：
 - i. SizeVT()：预计算消息二进制大小，避免运行时反射遍历字段；
 - ii. EqualVT()：直接比较消息字段值，跳过反射和类型检查；
 - iii. MarshalVT()：高效编码为二进制；
 - iv. UnmarshalVT()：快速解码，比proto.Unmarshal()性能高很多；
 - v. ReturnToVTPool()：集成对象池，复用消息实例减少内存分配；
 - vi. CloneVT()：深拷贝消息，避免反射复制；
 - d. 下面分别测试UnmarshalVT()和proto.Unmarshal()、MarshalVT()和proto.Marshal()两组方法，其结果如下：
 - vii. 序列化：

```
run benchmark | debug benchmark
43 v func Benchmark_1(b *testing.B) {
44     data, _ := os.ReadFile("../output.bin")
45     req := &fc_mall_feature.TransformRequest{}
46     proto.Unmarshal(data, req)
47     for n := 0; n < b.N; n++ {
48         if _, err := proto.Marshal(req); err != nil {
49             b.Fatal(err)
50         }
51     }
52 }
53
run benchmark | debug benchmark
54 v func Benchmark_2(b *testing.B) {
55     data, _ := os.ReadFile("../output.bin")
56     req := &fc_mall_feature.TransformRequest{}
57     req.UnmarshalVT(data)
58     for n := 0; n < b.N; n++ {
59         if _, err := req.MarshalVT(); err != nil {
60             b.Fatal(err)
61         }
62     }
63 }
64
65
66
```

问题 21 输出 调试控制台 终端 窗口 9

```
[root@drewjjli-1t70sepea fc_mall_feature]# go test -bench=. -benchmem ./test
goos: linux
goarch: amd64
pkg: git.woa.com/sgamesvr/sgameproto/sgameproto/full_control/fc_mall_feature/test
cpu: Intel(R) Xeon(R) Platinum 8255C CPU @ 2.40GHz
Benchmark_1-48      354      3337729 ns/op      323570 B/op      8269 allocs/op
Benchmark_2-48      396      2896970 ns/op      256740 B/op      4111 allocs/op
PASS
ok      git.woa.com/sgamesvr/sgameproto/sgameproto/full_control/fc_mall_feature/test  3.025s
[root@drewjjli-1t70sepea fc_mall_feature]#
```

viii. 反序列化：

```
run benchmark | debug benchmark
> 11 func Benchmark_1(b *testing.B) {
12     data, _ := os.ReadFile("../output.bin")
13     for n := 0; n < b.N; n++ {
14         req := &fc_mall_feature.TransformRequest{}
15         if err := proto.Unmarshal(data, req); err != nil {
16             b.Fatal(err)
17         }
18     }
19 }
20
run benchmark | debug benchmark
> 21 func Benchmark_2(b *testing.B) {
22     data, _ := os.ReadFile("../output.bin")
23     for n := 0; n < b.N; n++ {
24         req := &fc_mall_feature.TransformRequest{}
25         if err := req.UnmarshalVT(data); err != nil {
26             b.Fatal(err)
27         }
28     }
29 }
30
```

问题 21 输出 调试控制台 终端 窗口 9

```
• [root@drewjjli-1t70osepea fc_mall_feature]# go test -bench=. -benchmem ./test
goos: linux
goarch: amd64
pkg: git.woa.com/sgamesvr/sgameproto/sgameproto/full_control/fc_mall_feature/test
cpu: Intel(R) Xeon(R) Platinum 8255C CPU @ 2.50GHz
Benchmark_1-48      240      5202501 ns/op      1159869 B/op      22888 allocs/op
Benchmark_2-48      306      3604776 ns/op      1027951 B/op      14681 allocs/op
PASS
ok      git.woa.com/sgamesvr/sgameproto/sgameproto/full_control/fc_mall_feature/test    3.299s
○ [root@drewjjli-1t70osepea fc_mall_feature]#
```

从测试结果来看，UnmarshalVT()和MarshalVT()性能确实略有提升。

2. 将vtprotobuf用于fc_mall_feature服务；

```

1 // vtcodec.go
2 // 定义 VT 序列化方法接口
3 type VTMarshaler interface {
4     MarshalVT() ([]byte, error)
5 }
6
7 type VTUnmarshaler interface {
8     UnmarshalVT([]byte) error
9 }
10
11 // VTSerializer 实现 vtprotobuf 优化的序列化器
12 type VTSerializer struct {
13 }
14
15 // NewVTSerializer 创建 VT 序列化器实例
16 func NewVTSerializer() codec.Serializer {
17     return &VTSerializer{}
18 }
19
20
21 // vtSerialization.go
22 // Unmarshal 使用 UnmarshalVT 进行高效反序列化
23 func (s *VTSerializer) Unmarshal(in []byte, body interface{}) error {
24     if body == nil {
25         return errors.New("nil body for unmarshaling")
26     }
27     rv := reflect.ValueOf(body)
28     if rv.Kind() != reflect.Ptr || rv.IsNil() {
29         return errors.New("body must be a non-nil pointer")
30     }
31     // 检查body是否实现 VTUnmarshaler 接口
32     if unmarshaler, ok := body.(VTUnmarshaler); ok {
33         return unmarshaler.UnmarshalVT(in)

```

在主函数注册覆盖原来的`codec.SerializationTypePB`对应的序列化接口，如果新的pb结构实现了`UnmarshalVT()`和`MarshalVT()`方法就使用，如果没有就使用标准proto库的方法。要完成现网测试，还需要将sgameproto中的`fc_mall_feature`用`protoc-gen-go-vtproto`插件生成pb文件（即此时pb结构实现了`UnmarshalVT()`和`MarshalVT()`方法）。

现在只是把`codec.Serializer`的注册代码写好了，明日还需要联合sgameproto进行测试，并最终在现网环境观察对性能提升有多少。

明日待办：

1. 在已完成`codec.Serializer`的注册代码基础上，完成现网的测试，观察此方案对于性能优化有多少影响；